



Computer Networks

计算机网络 (CST2398)

2025-2026 学年第二学期 (26-Spring)

计算机科学与技术学院 王洁

wangjie_tongji@tongji.edu.cn



04

Data Link Control

Contents of this Chapter

- Functions of **Data Link Control (DLC)**

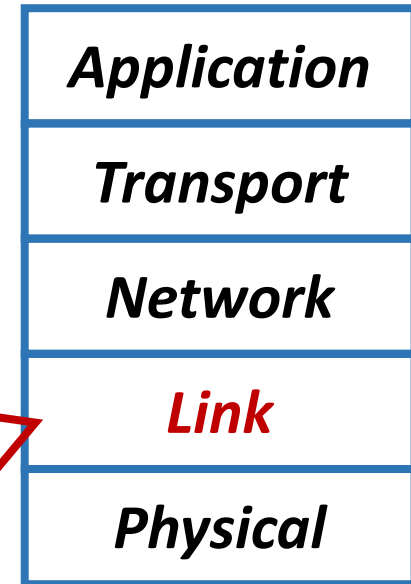
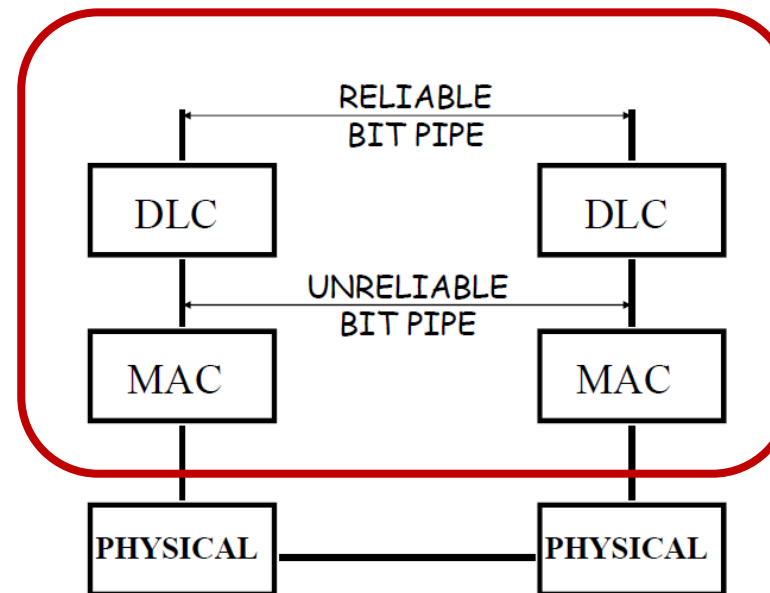
- 1) Framing

- 2) Error control:

- Error Control Coding
- Retransmission

- 3) Regulating data flow

- ARQ (Automatic Repeat reQuest)



DLC: Providing an **error-free** bit pipe.

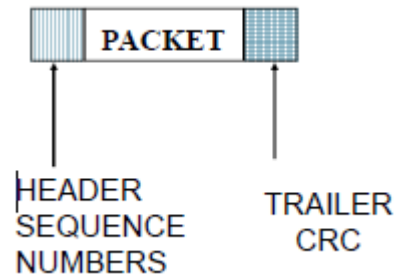
PHY: a bit pipe

Functions of the Data Link Layer

DLC: Providing an **error-free** bit pipe

- Providing service interface to the network layer

- **Framing (成帧)**



- Dealing with transmission errors -> **Error Control**

Error Control Coding

- Regulating data flow -> **Flow Control**

ARQ

to solve the problem of “a fast sender v.s. a slow receiver”

Services provided to the Network Layer (1/2)

1) **Unacknowledged (un-ACK'ed) connectionless service**

- No logical connection established
- Source sends independent frames w/o having the destination ACK them
- No detection/recovery for lost frames
- Suitable for reliable links (leave recovery to higher layers) e.g. LANs

BER: $< 10^{-10}$

2) **ACK'ed connectionless service**

- No logical connection established
- Each frame sent is individually ACK'ed
- Retransmit if timed out -> delay of a packet may be long
- Suitable for unreliable channels, e.g. wireless systems

BER: $< 10^{-9}$

Services provided to the Network Layer (2/2)

3) ACKed connection-oriented service

- *Connection is established* before any data are transferred
- *Frame is numbered and guaranteed to be received*
- *Each frame is received only once and in the right order*
- *Provides the network layer processes with a “reliable bit pipe”*

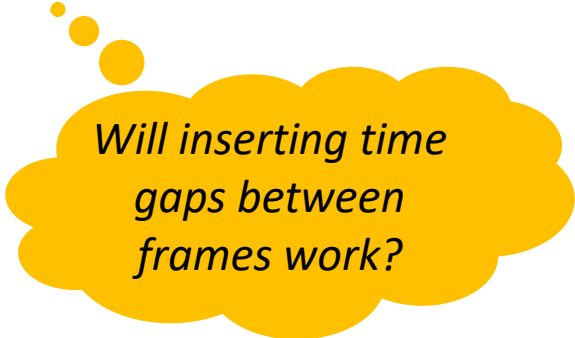


4.1

Framing

Framing

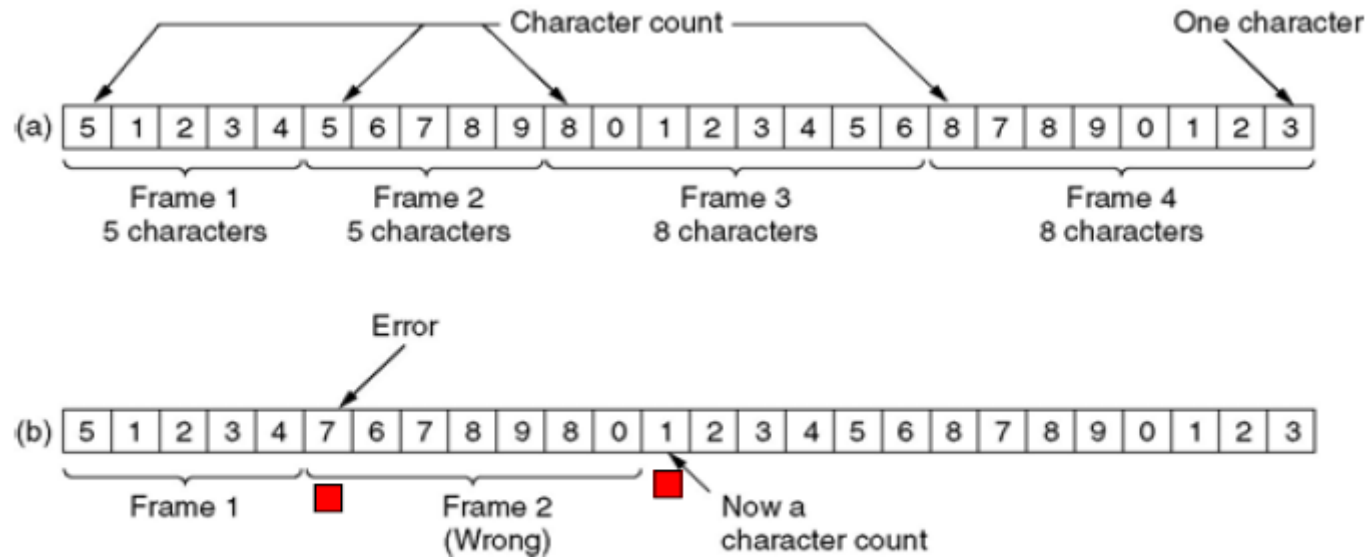
- *How does the receiver know where a frame starts and ends?*
 - *Character-based count*
 - ***Flag** bytes with **byte stuffing***
 - *Starting and ending **flags**, with **bit stuffing***



*Will inserting time
gaps between
frames work?*

Framing #1: Character-based count

- **Insert a header to specify the #(number) of characters in the frame**

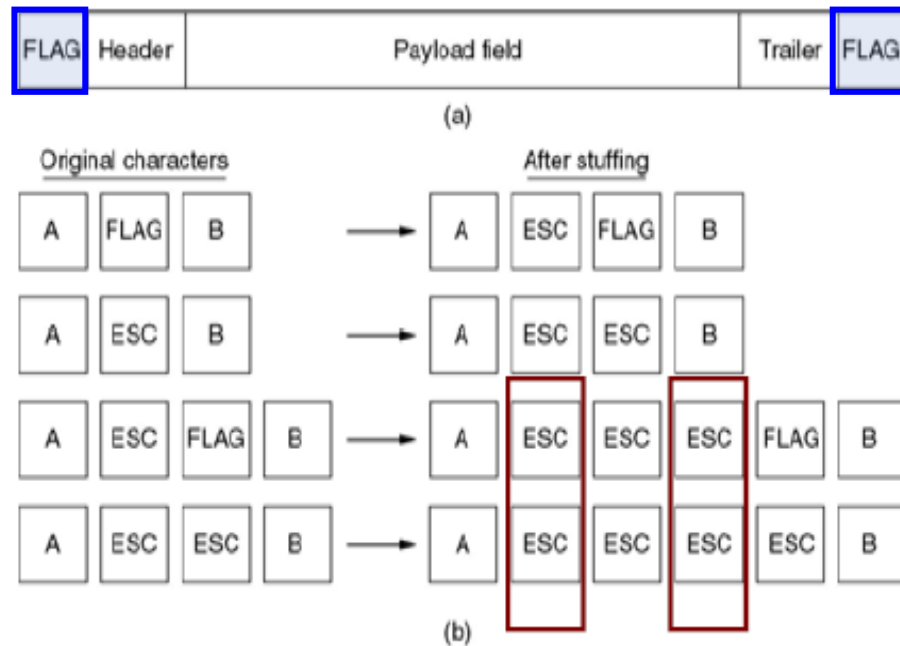


A character stream. (a) Without errors. (b) With one error.

Not good, why?

Framing #2: Flag bytes with byte stuffing

- Insert **FLAG** bytes to sperate frames
- If data has **FLAG** bytes, then insert a **ESC** byte before it as **byte stuffing**



Framing #3: Flag byte with bit stuffing

- Insert **FLAG** bytes to sperate frames

(HEX) 0x7E

“a flag byte” = 01111110

(a) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

(b) 0 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 1 1 1 1 0 1 0 0 1 0

Stuffed bits

Stuff a 0 bit
whenever there are
5 consecutive 1's

(c) 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

Bit stuffing

Delete “0” at the receiver

- (a) The original data.
- (b) The data as they appear on the line.
- (c) The data as they are stored in receiver's memory after de-stuffing.

In reality?
Combined!
Used in USB



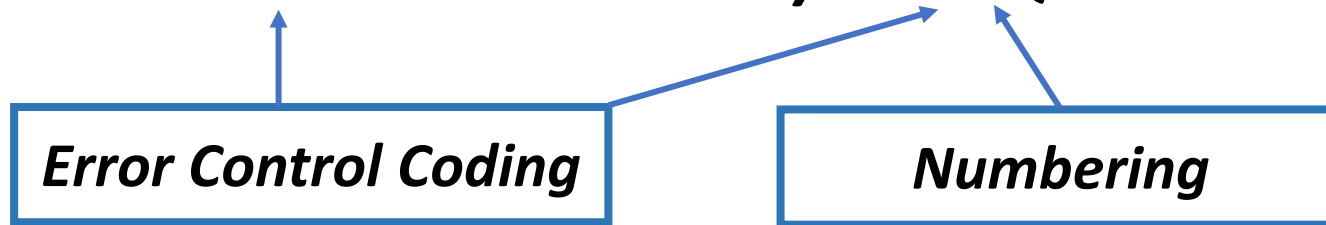
4.2

Error Control

Error Detection and Correction

- **DLC: unreliable bit pipe -> reliable bit pipe, how?**

- **Forward Error Correction and/or ARQ**



Forward Channel:
sender -> receiver

- **Error-Detecting Codes: Know if there is an erroneous bit**
 - **Parity Check (奇偶校验)**
 - **Cyclic Redundancy Check (CRC)**
- **Error-Correcting Codes: Know which bit is erroneous -> correct: Flip it!**
 - **Hamming Code (海明码)**

Definitions in ECC

- **Codeword:** n -bit unit ($n = m + r$) containing **m -bit data** and **r -bit redundant**, or **check bits**, is referred to as an n -bit codeword



- **The length of a Burst of Errors:** the number of **consecutive** bits from the first error bit to the last error bit.
- **Burst Error Detecting Capability** (of a code): the largest integer B by which a code can detect all the bursts of length B or less.
- **The Minimum Distance of the Code:** the smallest number of errors that can convert one codeword into another.

Hamming Distance

- **Hamming Distance (海明距离) d** : the number of bit positions in which two legit codewords differ
 - To Calculate: **Exclusive OR (XOR)** 异或 **mod 2 sum** 半加器

$$\begin{array}{r} 10001001 \\ \oplus 10110001 \\ \hline 00111000 \end{array}$$

0 for same values
1 for different values

- **Meaning**: It measures the resistance of codewords against bit errors.
 - If **two codewords** have Hamming distance **d** , then **d single-bit errors** can convert one codeword to another.
 - E.g. A min. of 3-bit error can turn codeword 10001001 to 1011001.
- For coding scheme: the **min. Hamming distance** between any pair of codewords.

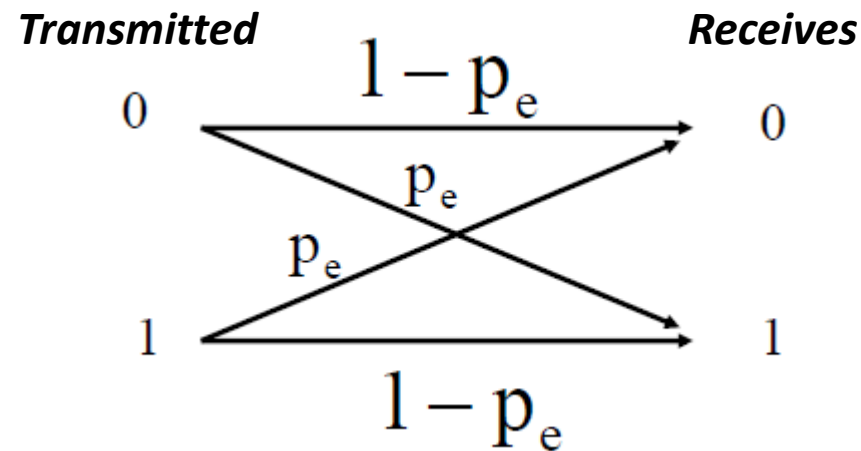
Bit Error: Binary Symmetric Channel Model

- **How does a bit error happen? A bit is flipped.**

- Transmitted 1 -> receives 0, or
- Transmitted 0 -> receives 1

$$p_{1,0} = p_{0,1} = p_e$$

- To model this, we have **Binary Symmetric Channel**



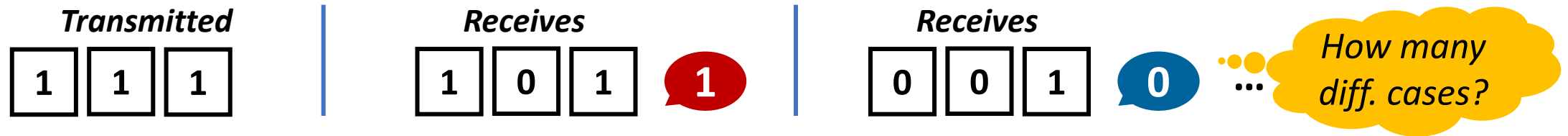
Bit errors are assumed to be independent

- **Bit Error Rate (BER) p_e : the probability that a bit is flipped.**

Ex.3.1 Bit Error Rate of Majority Vote

Consider an error correction coding scheme called Majority Vote.

- Each bit is repeated and transmitted 3 times.
- Receiver receives 3 times, and takes a majority vote to determine transmitted bit.



- m, r, n ? Codewords? What is the Hamming distance between the two codewords?

Suppose the bit flipping probability of a binary symmetrical channel is p_e ,

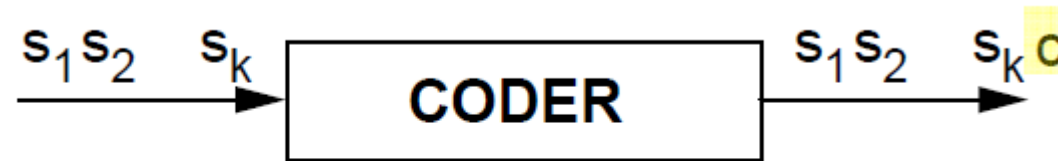
- 1) What is the probability of a bit error (**Bit Error Rate, BER**)?
- 2) What is the probability of a packet error (**Packet Error Rate, PER**), which is N bits long?

What is the cost?

Single Parity Code

- **Parity Check: append a single bit to the data.**
 - If the number of 1 bits in the codeword is **even**, a bit **0** is appended.
 - If the number of 1 bits in the codeword is **odd**, a bit **1** is appended.

The total #
of 1's?



XOR,
mod 2 sum

where $c = s_1 \oplus s_2 \oplus \dots \oplus s_k$, the modulo-2 sum

- **What is the check bit and codeword for 1011010?**
- **The minimum distance of the single parity code?**

Can it **correct**
errors?

Horizontal and Vertical Parity Checks

- **2-D check:** A two-dimensional **array** with 1-bit parity check for each row and 1-bit check for each column

$$\begin{bmatrix} s_{11} & s_{12} & s_{13} & \cdots & s_{1k} & c_1 \\ s_{21} & s_{22} & s_{23} & \cdots & s_{2k} & c_2 \\ s_{31} & s_{32} & s_{33} & \cdots & s_{3k} & c_3 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ s_{n1} & s_{n2} & s_{n3} & \cdots & s_{nk} & c_n \\ r_1 & r_2 & r_3 & \cdots & r_k & c_{n+1} \end{bmatrix}$$

What is Hamming distance between two codewords?

$$c_i = s_{i1} \oplus s_{i2} \oplus s_{i3} \oplus \cdots \oplus s_{ik} \quad \text{Horizontal check}$$

$$r_j = s_{1j} \oplus s_{2j} \oplus s_{3j} \oplus \cdots \oplus s_{nj} \quad \text{Vertical check}$$

Ex.3.2 Parity Check and its Detection Capacity

Find the horizontal and vertical parity check bits for the following data.

1	0	0	1	0	1	0
0	1	1	1	0	1	0
1	1	1	0	0	0	1
1	0	0	0	1	1	1
0	0	1	1	0	0	1

What if the circled bits are flipped? Can they be **detected**? Can they be **corrected**?

1	0	0	1	0	1	0
0	1	1	1	0	1	0
1	1	1	0	0	0	1
1	0	0	0	1	1	1
0	0	1	1	0	0	1

1	0	0	1	0	1	0
0	1	1	1	0	1	0
1	1	1	0	0	0	1
1	0	0	0	1	1	1
0	0	1	1	0	0	1

1	0	0	1	0	1	0
0	1	1	0	1	0	0
1	1	1	0	0	0	1
1	0	0	0	1	1	1
0	0	1	1	0	0	1

1	0	0	1	0	1	0
0	1	1	0	1	0	0
1	1	1	0	0	0	1
1	0	0	0	1	1	1
0	0	1	1	0	0	1

What is max #
of flips that can
still be detected?

Observations from Parity Check

- To **detect d** errors: code distance must be $\geq d+1$
 - **WHY:** d single-bit error cannot change a codeword to another codeword
- To **correct d** errors: code distance must be $\geq 2d+1$
 - **WHY:** d single-bit error happens, the original codeword is still the closest one.
- E.g. there are 4 valid/legit codes: 0000000000, 0000011111, 1111100000, 1111111111. Code distance 5.
 - **Detect:** $d+1 = 5 \Rightarrow d=4$, can detect **every** 4-bit error.
 - **Correct:** $2d+1 = 5 \Rightarrow d=2$, can correct **every** 2-bit error.

Can you decode
"0000000111"?
2-bit? 3-bit?

Recall Hamming Distance

- **Hamming Distance (海明距离) d** : the number of bit positions in which two codewords differ

- To Calculate: **Exclusive OR (XOR)** 异或 **mod 2 sum** 半加器

$$\begin{array}{r} 10001001 \\ \oplus 10110001 \\ \hline 00111000 \end{array}$$

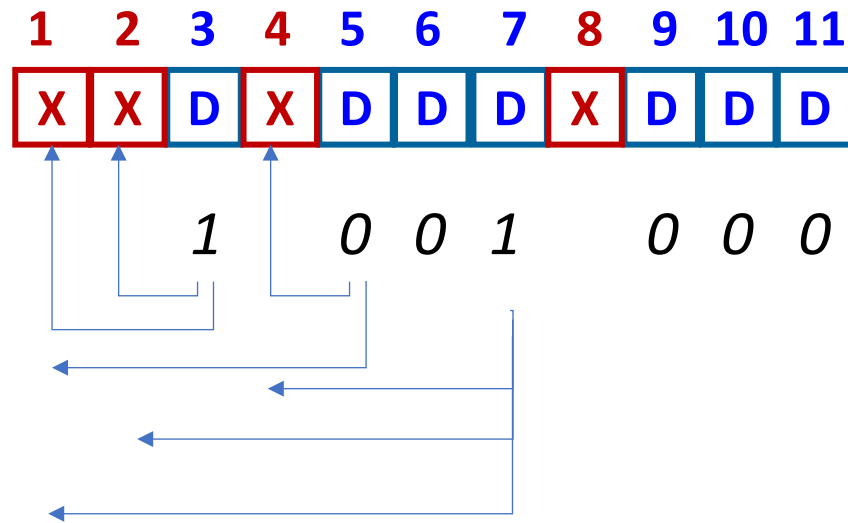
0 for same values
1 for different values

Differentiate: 2
codewords v.s. a
coding scheme

- **Meaning**: It measures the resistance of codewords against bit errors.
 - If **two codewords** have Hamming distance **d** , then **d single-bit errors** can convert one codeword to another.
 - E.g. A min. of 3-bit error can turn codeword 10001001 to 1011001.
- For a coding scheme: the min. Hamming distance between any pair of codewords.

Hamming Code

- **Error-Correcting Code:** detect + locate
- **Codeword:** r *check* bits (**X**) + m *data* bits (**D**)



correct d -bit
errors: distance
 $\geq 2d+1$

Encoding:

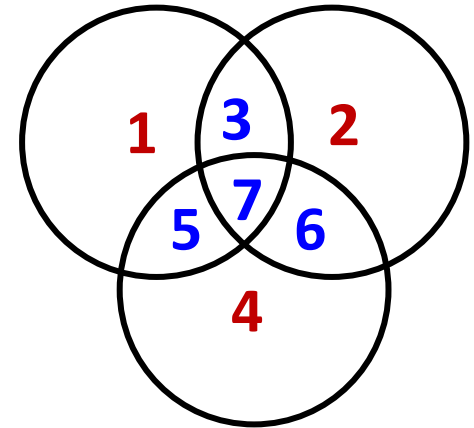
- 1) Place the **data bits**: 1001000
- 2) Calculate the **check bits**: any data bit (at location k) counts toward $(k)_{10} \rightarrow (k)_2$ locations in the check bits. Even parity.

- **Key Idea:** a data bit $\rightarrow$ different check bits
- What is the relationship between m and r ?

(7,4) Hamming Code

- $m=4, r=3 \rightarrow n=7$: 4 **data** bits + 3 **check** bits = 7-bit codeword
 - $3 = 1+2; 5 = 1+4; 6 = 2+4; 7 = 1+2+4$
 - **Rule:** Each circle/row must have even parity (mod 2 sum = 0).

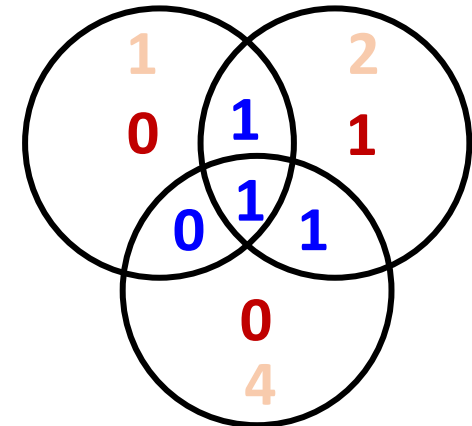
1	2	3	4	5	6	7		
X	X	D	X	D	D	D	Rule	Check bit contributed
X	-	D	-	D	-	D	Even Parity	1
-	X	D	-	-	D	D	Even Parity	2
-	-	-	X	D	D	D	Even Parity	4



Example: (7,4) Hamming Code-Encoding

- **Rule: Each circle/row must have even parity.**
- **E.g. Data = 1011, Codeword = ?**

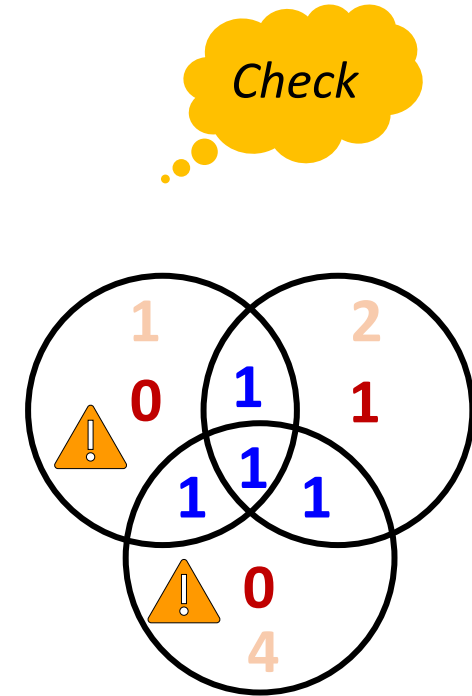
1	2	3	4	5	6	7		
X	X	1	X	0	1	1	Rule	Check bit contributed
X	-	1	-	0	-	1	$X=1\oplus 0\oplus 1=0$	1
-	X	1	-	-	1	1	$X=?$	2
-	-	-	X	0	1	1	$X=?$	4



Example: (7,4) Hamming Code-Detection

- **Rule: Each circle/row must have even parity.**
- **E.g. Received 0110111, then?**

1	2	3	4	5	6	7		
0	1	1	0	1	1	1	Rule: Even Parity	Compute Parity
0	-	1	-	1	-	1	$0 \oplus 1 \oplus 1 \oplus 1 =$	1 -> Not OK
-	1	1	-	-	1	1	$1 \oplus 1 \oplus 1 \oplus 1 =$	0 -> OK
-	-	-	0	1	1	1	$0 \oplus 1 \oplus 1 \oplus 1 =$	1 -> Not OK



- **Detection result: the bit at location $(101)_2 = (5)_{10}$ is wrong!**

Ex.3.3 Hamming Code Detection

Receiving 0100011, knowing there is a 1-bit error. Correct it.

1	2	3	4	5	6	7		
0	1	0	0	0	1	1	Rule: Even Parity	Compute Parity
	-		-		-			
-			-	-				
-	-	-						

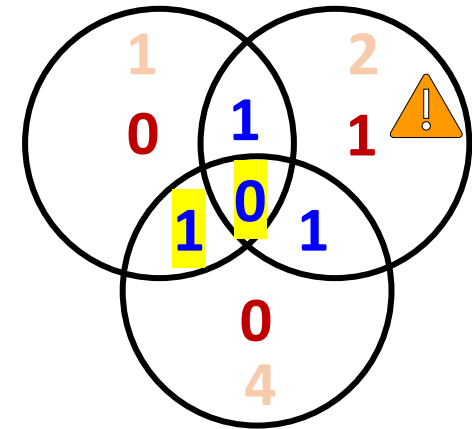


What about 2-bit error? Not OK.

- **Rule: Each circle/row must have even parity.**
- 2-bit error occurred, sending 0110011, received 0110**110**

Can still tell something is wrong

1	2	3	4	5	6	7		
0	1	1	0	1	1	0	Rule: Even Parity	Compute Parity
0	-	1	-	1	-	0	$0 \oplus 1 \oplus 1 \oplus 0 =$	0 -> OK
-	1	1	-	-	1	0	$1 \oplus 1 \oplus 1 \oplus 0 =$	1 -> Not OK
-	-	-	0	1	1	0	$0 \oplus 1 \oplus 1 \oplus 0 =$	0 -> OK



- Detection result: the bit at location $(010)_2 = (2)_{10}$ is wrong

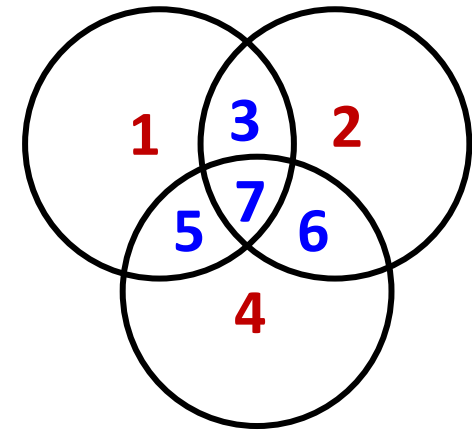
Incorrect!!!

Summary of Hamming Code

- Can **detect and correct 1-bit error**.
- Can **detect 2-bit error**, but can not correct it.
- **WHY:** The min. Hamming distance between any two valid codewords in the (7,4) Hamming code is **3**. ... Why?
- **Cost:** added $r=3$ check bits to $m=4$ data bit.

Compare to
single parity
check?

How about
bursty errors?



Cyclic Redundancy Check (CRC)

- **Cyclic Redundancy Check (CRC) 循环冗余校验**
 - **Basic idea:** generate parity checks on different subsets of the bit string.
 - Most-commonly used: **Polynomial code 多项式码**
 - **Advantage:**
 - fast calculation, low cost
 - stronger detection (compared to single parity check)
 - **Application:** widely used in computer communication, e.g. Ethernet, USB, compression, video coding, image storage, I/O

Polynomial Code

- ***k-bit*** string -> a ***k-term*** polynomial with coefficients of 0 and 1

- E.g. 1011 -> $1 \cdot x^3 + 0 \cdot x^2 + 1 \cdot x + 1 \cdot x^0 = x^3 + x + 1$

$k = 4$

x^{k-1}

Degree k-1

- *k* data bits: $m_{k-1}, m_{k-2}, \dots, m_1, m_0$
- Polynomial representation of the *k*-bits data:

$$M(x) = m_{k-1} \cdot x^{k-1} + m_{k-2} \cdot x^{k-2} + \dots + m_1 \cdot x + m_0$$

Basic Idea of Polynomial CRC

- Sender and Receiver agree on a **generator polynomial, $G(x)$** of degree r

$$G(x) = x^r + g_{r-1} \cdot x^{r-1} + \cdots + g_1 \cdot x + 1$$

- $G(x)$ has at least 2 **non-zero terms**, i.e., x^r and 1

- **Rule:** For a message/data $M(x)$,

- **Remainder** $r(x) = \text{Remainder}\left[\frac{M(x) \cdot x^r}{G(x)}\right]$

- Modulo 2 arithmetic

- The remainder is of degree (at most) $r - 1$.

- **CRC Polynomial** $T(x) = M(x) \cdot x^r - r(x) = M(x) \cdot x^r + r(x)$

- Check with the generator polynomial $G(x)$:

- $T(x)$ must be (modulo 2) divisible by $G(x)$ if there is no error.

- If there is an error, $T(x)$ will not be (modulo 2) divisible by $G(x)$.

Mod 2,
how?

Encoding

Detection

Code-
polynomial

Example: CRC Encoding

- E.g. $r=3$, $k=3$, data 101

- Generator polynomial

$$G(x) = x^3 + x^2 + 1$$

- Data Polynomial

$$M(x) = x^2 + 1$$

- Remainder

$$r(x) = x^2 + x$$

- Code/CRC Polynomial

$$T(x) = M(x) \cdot x^r + r(x) = x^5 + x^3 + x^2 + x$$

$$\begin{array}{r}
 \overline{x^2 + x} \\
 x^3 + x^2 + 1 \overline{) x^5 + x^3} \\
 \underline{x^5 + x^4 + x^2} \\
 x^4 + x^3 + x^2 \\
 \underline{x^4 + x^3 + x} \\
 x^2 + x \longrightarrow \text{Remainder}
 \end{array}$$

Ex.3.4 CRC

For data $M = 1101011011$, $G(x) = x^4 + x + 1$

- 1) What is r and k ?
- 2) What is the message/data polynomial $M(x)$?
- 3) What is the code/CRC polynomial $T(x)$?

Error Detection with CRC

- The *received message* $Y(x)$ can be modeled as

$$T(x) \rightarrow \text{Channel} \rightarrow Y(x) = T(x) + E(x)$$

$$\begin{aligned} \text{Remaider} \left[\frac{Y(x)}{G(x)} \right] &= \text{Re mainder} \left[\frac{T(x)}{G(x)} + \frac{E(x)}{G(x)} \right] \\ &= \text{Re mainder} \left[\frac{E(x)}{G(x)} \right] \end{aligned}$$

- Then
 - If no error occurs, $E(x) = 0 \rightarrow \text{Remainder} \left[\frac{E(x)}{G(x)} \right] = \text{Remainder} \left[\frac{Y(x)}{G(x)} \right] = 0$
 - If there is an error, $E(x) \neq 0$, there are two possible cases:
 - $\text{Remainder} \left[\frac{Y(x)}{G(x)} \right] \neq 0 \rightarrow$ **Error is detectable.**
 - $\text{Remainder} \left[\frac{Y(x)}{G(x)} \right] = 0$, i.e., $E(x) = G(x)z(x)$, $z(x)$ is non-zero polynomial $\rightarrow$ **NOT Detectable**

CRC Property 1. Single Error

- **Single error:** $E(x) = x^j$ for some j
- $G(x)$ has at least 2 non-zero terms, x^r and 1, then:
 - For any $z(x) \neq 0$, $G(x)z(x)$ must have at least 2 non-zero terms.
 - $E(x) = x^j$ is not divisible by $G(x)$.
 - **All single errors can be detected!**

NOT Detectable if.f.

$\text{Remainder}\left[\frac{E(x)}{G(x)}\right] = 0, \text{ i.e.,}$

$$E(x) = G(x)z(x)$$

CRC Property 2. Double Error

- **Double error:** $E(x) = x^i + x^j = x^j(x^{i-j} + 1)$
- Just showed in Property 1, x^j is not divisible by $G(x)$, now consider $x^{i-j} + 1$:
 - If $G(x)$ is carefully chosen, it can be shown that $x^{i-j} + 1$ will not be divisible by $G(x)$, for large $k=i-j$.
 - E.g. $x^k + 1$ is not divisible by $G(x) = x^{15} + x^{14} + 1$ for any value of k smaller than 32768.

NOT Detectable if.f.

$\text{Remainder}\left[\frac{E(x)}{G(x)}\right] = 0, \text{ i.e.,}$

$$E(x) = G(x)z(x)$$

- **Double errors can be detected!**

CRC Property 3. Odd # of Errors

- In most implementations: $G(x) = (x + 1)Poly(x^{r-1})$, then
- Any error $E(x)$ containing **odd #** of non-zero coefficients will not be divisible by $(x + 1)$ -> It will not be divisible by $G(x)$.
- **Any odd # of errors can be detected!**
- Burst Error of length $\leq r$ can be detected. (Read the textbook for detailed explanation.)

CRC in the Real-world Applications

- **Standard CRC Polynomials**

- $CRC-16 \ G(x) = x^{16} + x^{15} + x^2 + 1$

- $CRC-ITU-T \ G(x) = x^{16} + x^{12} + x^2 + 1$

- $IEEE \ 802 \ G(x) = x^{32} + x^{26} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + 1$



4.3

ARQ: Automatic Repeat reQuest

Dealing with transmission errors

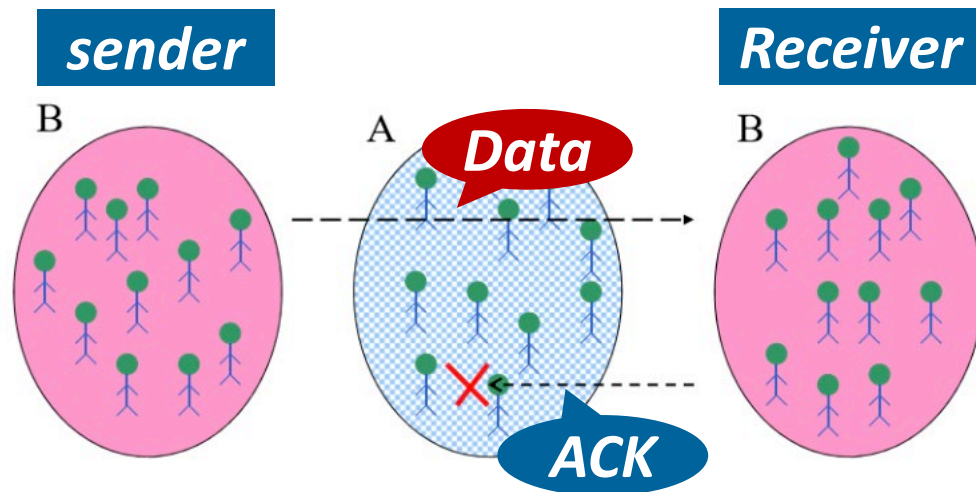
- *Error happens...*
 - *Control: detection / correction*
 - *Redundancy - higher overhead*
- *That's not enough.*
 - *Error correction has high overhead, e.g. the 7,4-Hamming code*
 - *But detection is not enough, so we need*
 - **Acknowledgement** - to tell the sender “something is (not) ok”
 - **Error recovery** - the sender needs to do something (e.g. retransmission)

Efficiency: 4/7,
too low!

ARQ

When error happens

- **The Three Armies Problem**



- Error can happen to the *data*.
sender -> receiver
- Error can also happen to *ACK*.
receiver -> sender

- **Observations:**

- *ACKs are needed.*
- *But ACK itself may not be properly delivered as well.*
- *Need an implicit/default action.*
e.g. consider data as not delivered when ACK is not received in 10 seconds.

ARQ Error Control

- **Most error control schemes are based on two schemes:**
 - 1) **Error Detection:** e.g. Parity check, CRC
 - 2) **Retransmission** (of erroneous frames)
- **Types of Errors: lost or damaged frame**
- **Automatic Repeat reQuest (ARQ) is such an error control scheme.**
 - **Correctness:** Does the protocol ensure that each frame is delivered (to the upper layer) once, only once, and error-free? *Yes or No*
 - **Efficiency:** How much of the bit-transmitting capacity of the bit-pipe is actually used to transmit data? *a ratio between 0 and 1*

And it can be used in
flow control as well.

Approach

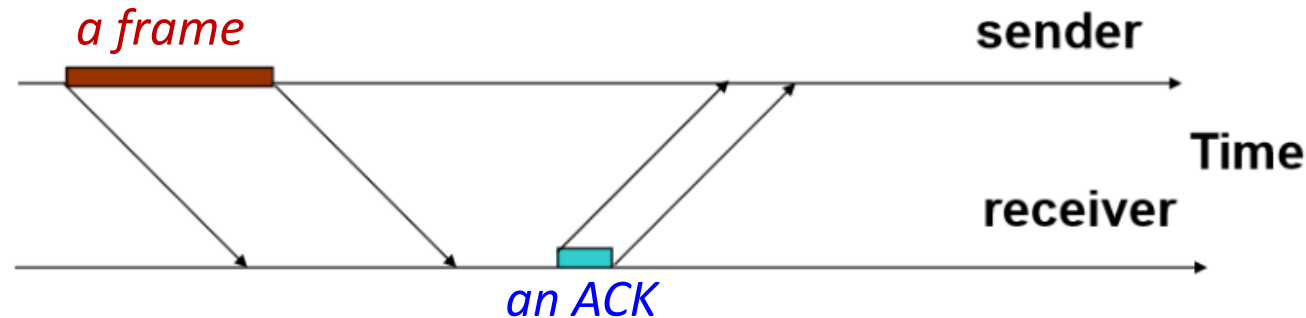
- **Sequence Numbers**

- a stream of bits -> a sequence of frames
- **Successive** frames have different id's (**seq. #**)

Why not require all frames to have different id's? -> **Reuse seq. #**

- **Timing Diagram**

- showing a sequence of events as time goes
- transmission of frames (start and end)
- delays



Frequently Used ARQ Protocols

- **Stop-And-Wait (SAW)**

- transmit -> *wait for the ACK* -> carry on
- “half-duplex” because of ACK

ACK for the
current frame

- **Go-Back-N (GBN)**

- sometimes called the “*sliding window*” 滑动窗口
- increases utilization (compared to SAW)

ACK for **multiple**
frames

- **Selective Repeat Protocol (SRP)**

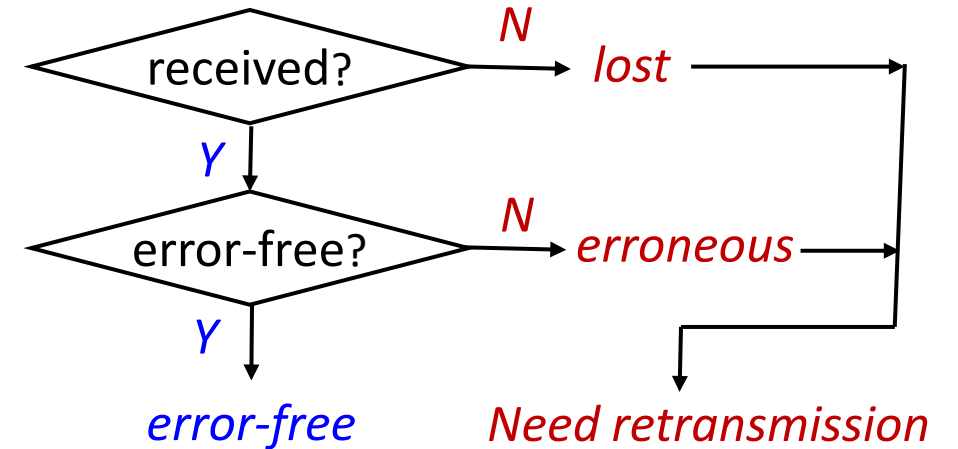
- (further) increases efficiency

Assumptions

- **Frames in DLC are either**

- *error-free*, or
- *erroneous*, or
- *lost*: some frames may never arrive

DLC of the **receiver**
can not see them



- **Errors can always be detected** - the receiver can always distinguish erroneous frames from error-free frames.
- **Each frame is subject to an arbitrary delay.**

True?
Hint: Hamming
distance and
detection

Stop-And-Wait (1/5)

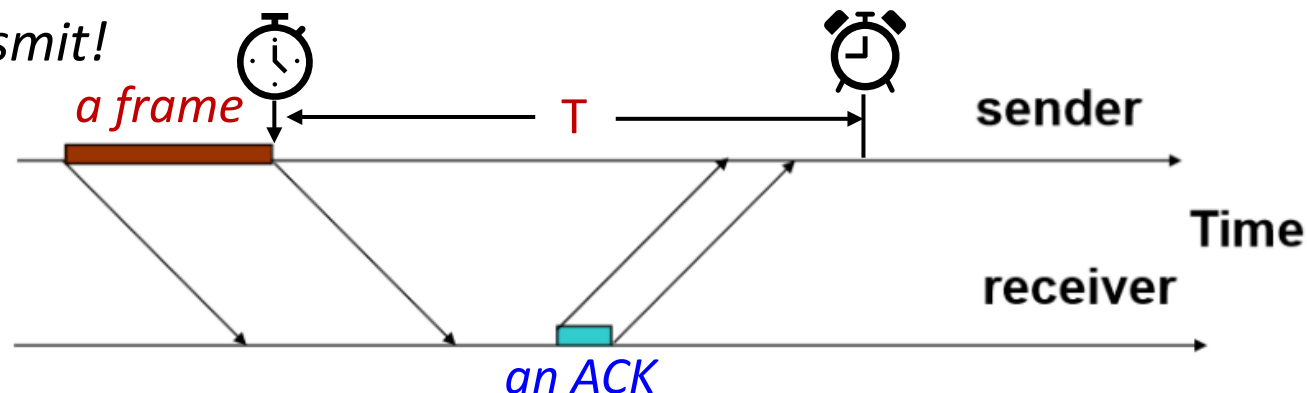
- **The simplest scheme:**

- The sender sends *one* frame and then *waits for an ACK* before proceeding.
- Can not wait forever -> A *timer* is kept for each frame at the sender.

- the *time-out* value T : $\geq$ a Round-Trip Time (RTT)

- This is an **implicit** request for retransmission:

- **Sender:** If no ACK is received and sender timer timed out -> something is wrong
-> retransmit!

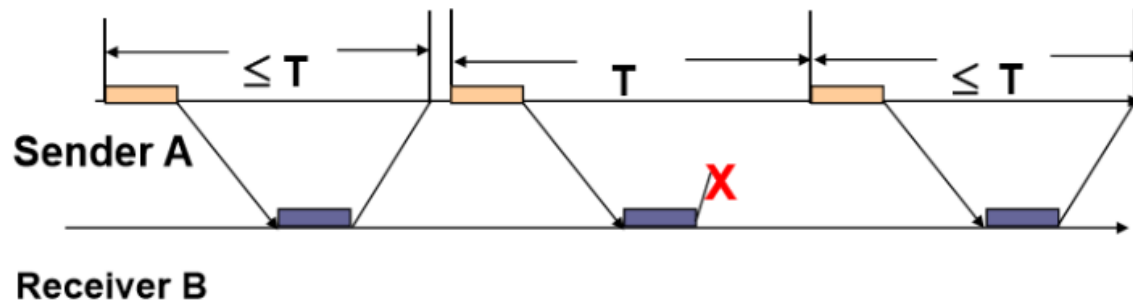


Stop-And-Wait (2/5)

- **Procedure of the simplest scheme:**

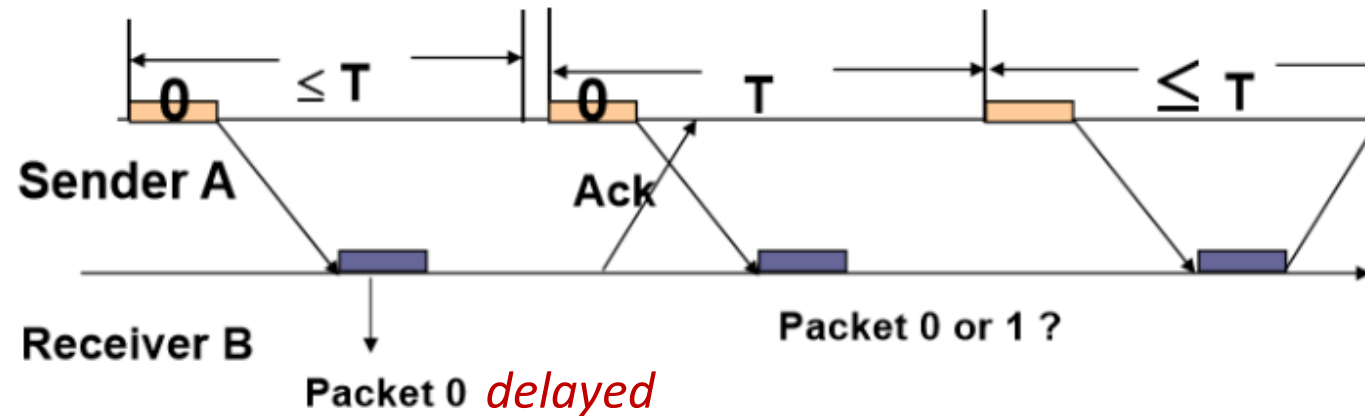
- Frames are transmitted from A to B.
 - A: start a **timer** of time-out value **T**
- The frame is received by the receiver B:
 - If the frame is error-free, B sends an **ACK** back to A.
 - If the frame is erroneous, B sends a negative ACK (**NAK**) back to A.

Note: ACK or NAK is also protected with CRC, because error can happen to it as well.



Stop-And-Wait (3/5)

- **Problem 1: Unnumbered frames**



- **Solution:**

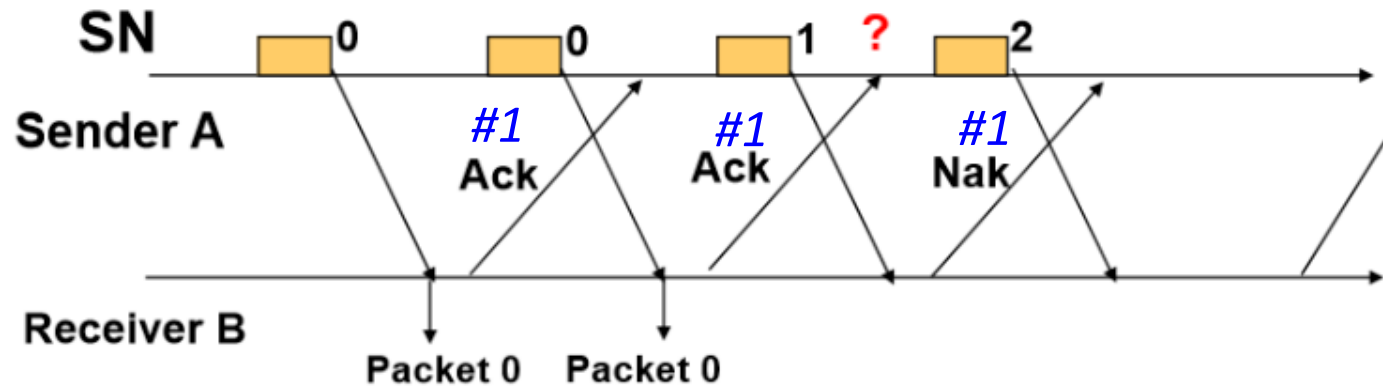
Use a **seq. #** in the frame header to identify **successive** frames.

- ACK for frame #0 is abnormally delayed.
- A retransmits frame #0, but B can not tell whether this frame is #0 or #1 (because there is no seq. #).

Note: Packets and frames are used interchangeably.

Stop-And-Wait (4/5)

- **Problem 2: Unnumbered ACKs**



- *Timer for frame #0 timed out, so A sends #0 twice.*
- *B recognizes frame #0 is being repeated, but A can not tell whether the ACK is for a repeated ACK for #0 or a new ACK for #1 (because there is no seq. # for the ACK).*

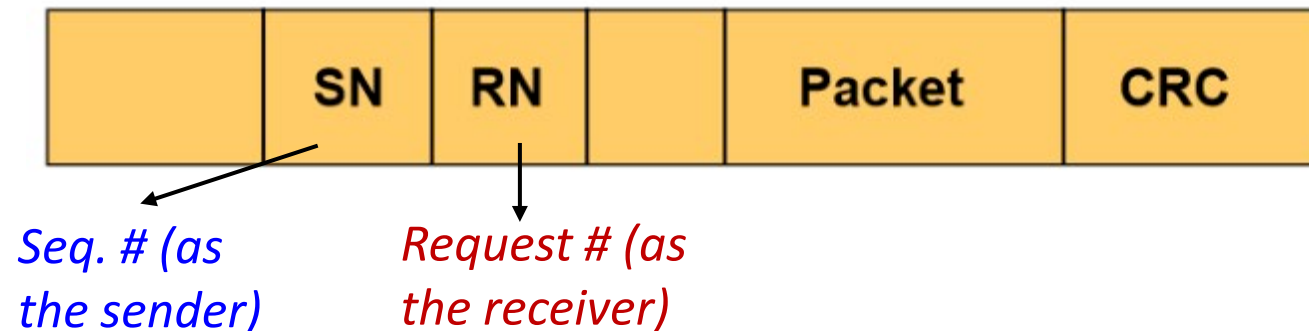
- **Solution:**

*Instead of returning ACK or NAK, the receiver returns **the # of the next frame** (called the **Request #**) it is waiting for.*

What about
NAK?

Stop-And-Wait (5/5)

- **Piggybacking: data transmission in both directions.**
 - In many cases, A needs to transmit data to B, and B also needs to transmit to A. A/B is both the sender and the receiver.
 - So receiver B defers the ACK and **piggybacks** the ACK (of the received data frame from A) in the **header of its next data frame** to A.
 - Similarly, A piggybacks its ACK in the header of its next data frame to B.



Detailed Algorithms of SAW

• Node A in the A-to-B Transmission • Node B in the A-to-B Transmission

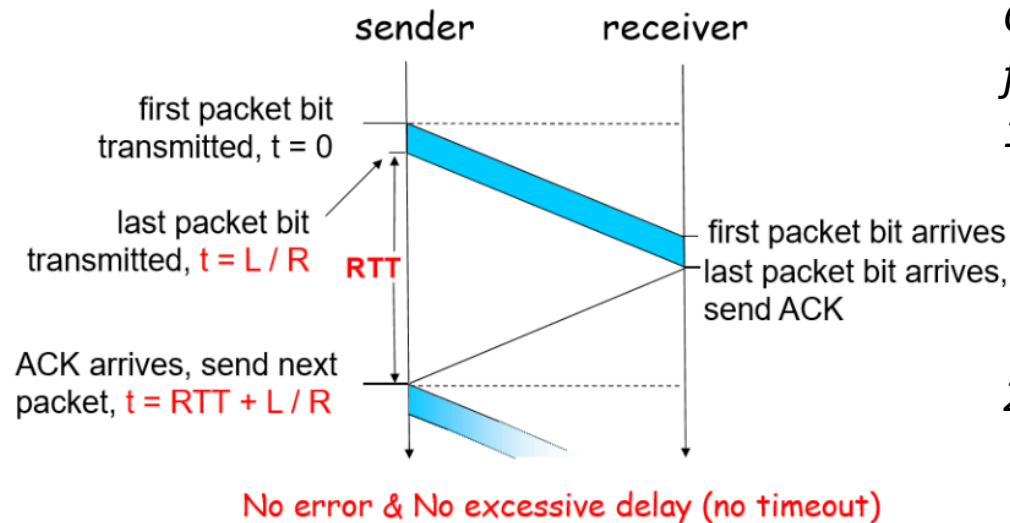
1. Set the integer variable **SN** to 0
2. Accept a packet from the next higher layer at A; if no packet is available, wait until it is; assign number **SN** to the new packet.
3. Transmit the **SN-th** packet in a frame containing **SN** in the sequence number field.
4. If an error-free frame is received from B containing a request number **RN** greater than **SN**, increase **SN** to **RN** and go to step 2.
5. If no such frame is received within some finite delay (timeout **T**), go to step 3.

1. Set the integer variable **RN** to 0 and then repeat steps 2 and 3 forever.
2. Whenever an error-free frame is received from A containing a sequence number **SN** equal to **RN**, release the received packet to the higher layer and increment **RN**.
3. At arbitrary times, but within bounded delay after receiving any error-free data frame from A, transmit a frame to A containing **RN** in the request number field.

KEY: Timer,
SN, and RN

Efficiency of the SAW

- **Assume:**
 - fixed frame size, infinite seq. #, independent frame errors, infinite retransmissions.
- **Efficiency = Throughput / Capacity**
 - measured in the amount of bits transmitted, or the time spent on transmission



Given : timeout T , frame size L , data rate R , RTT S , and frame error probability $1-p$.

1) No error, no timeout case:

- One frame every $S + L/R$ s
- Time spent on data transmission L/R s
- Efficiency $\eta = \frac{L/R}{S + L/R}$

2) Generally with independent errors:

$$\eta = \frac{L/R}{S + \frac{L}{R} + T(1-p)/p}$$

Different scenarios?

Ex.3.5 SAW Efficiency

Consider a satellite channel, upon which a SAW protocol is used.

- Data rate $R = 50 \text{ kbps}$
- Round-trip time (RTT) $S = 500 \text{ ms}$ (i.e. propagation delay for each hop is 250 ms)
- Frame size $L = 1000 \text{ bits}$

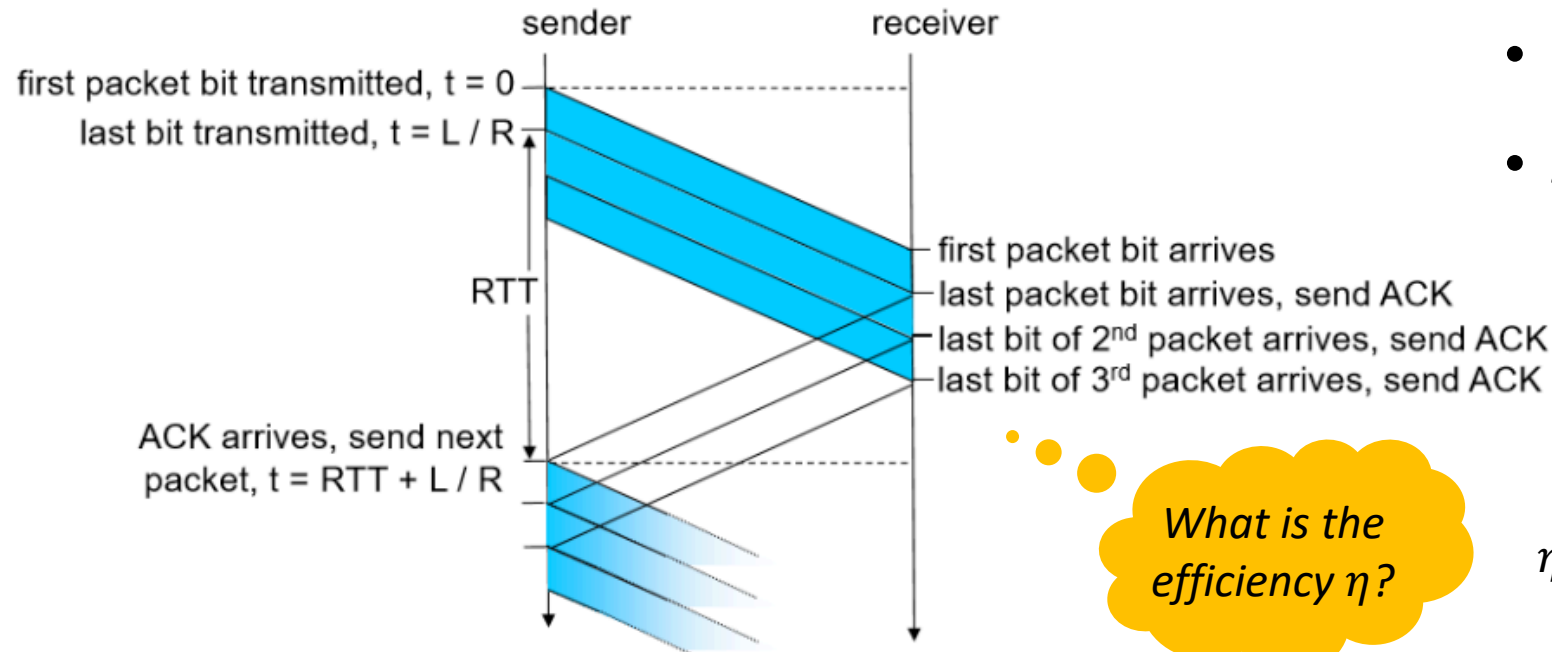
Assume there is no queuing delay or processing delay.

- 1) When will the first ACK be received by the sender, assuming there is no error?
- 2) What is the efficiency of this SAW?
- 3) During this send-receive process, how long will the sender be idle?
- 4) What if it is an Ethernet channel, with RTT $S = 10 \text{ ms}$?

How to improve the efficiency?

Improvement: Pipelining

- **Pipelining: sender allows multiple, yet-to-be-ACKed frames**
 - Requirement: **buffer** at the sender and receiver side
 - Each time **ACK for a range of frames**



What is the efficiency η ?

- **Pipelined Protocols**

- **Go-Back-N**
- **Selective Repeat**

$$\eta = \frac{3L/R}{S + 3L/R} > \frac{L/R}{S + L/R}$$

Sliding Window Protocols

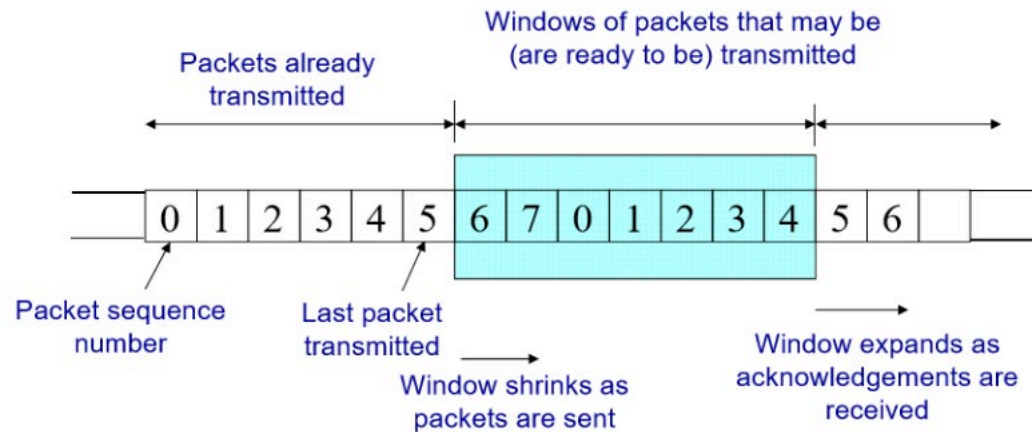
- **Drawbacks of SAW:** inefficiency due to the “wait”, one frame each time
- **Assume there is no error (no delay and no loss), sliding window:**
 - allows the transmission of multiple frames
 - assigns each frame a k -bit seq. #
 - The range of the seq. # is $[0, 1, \dots, 2^k - 1]$.
 - Packets are counted modulo 2^k .
 - is basically a flow control protocol (because of no error)
- **With errors/loss -> Go-Back-N and Selective Repeat**

Sliding Window Operations

- **Piggybacking and Two Windows**

- **Sending Window**

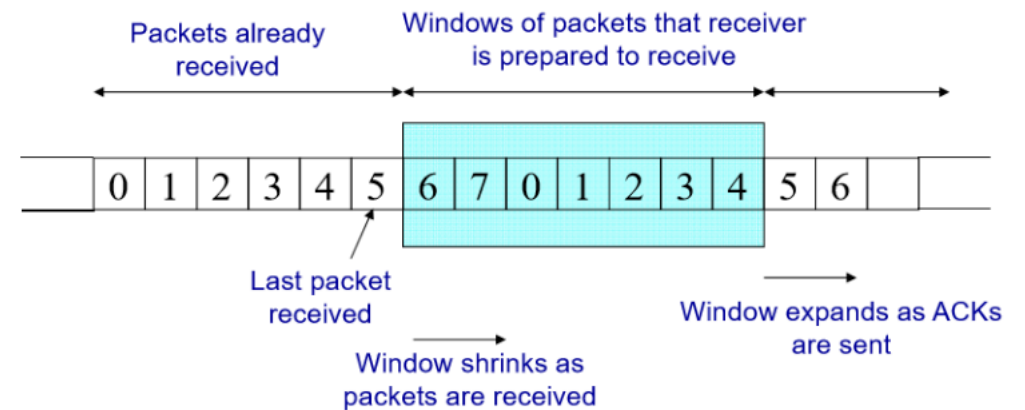
- Seq. # within the sender's window represent the frames that are **ready to be sent**, but not ACKed.



Assume no error, no loss, only delay.

- **Receiving Window**

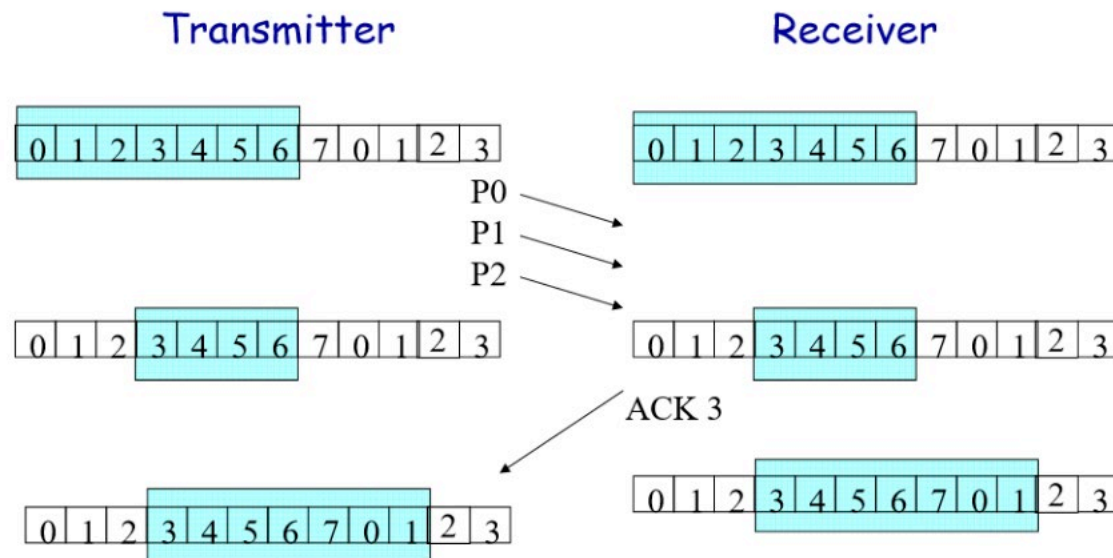
- Seq. # within the receiver's window represent the frames that are **permitted to accept**.



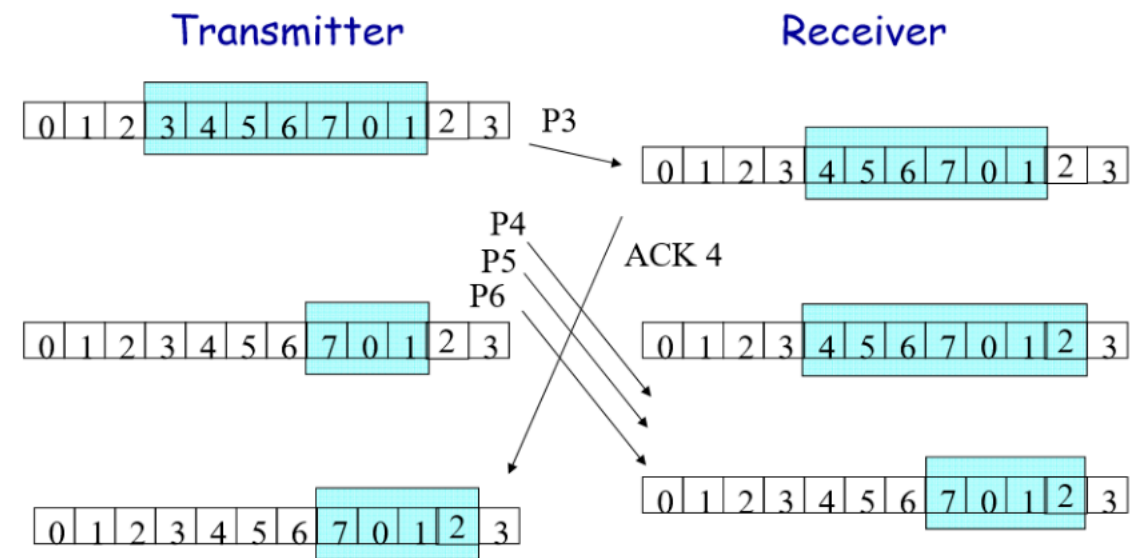
Note: In reality, the window only contains seq. #s (or address to where the actual frame data is stored), so the implementation does not need to handle frames of different lengths.

E.g. Sliding Window Operations

- Sender sends P0, P1, P2
- Receiver sends ACK 3



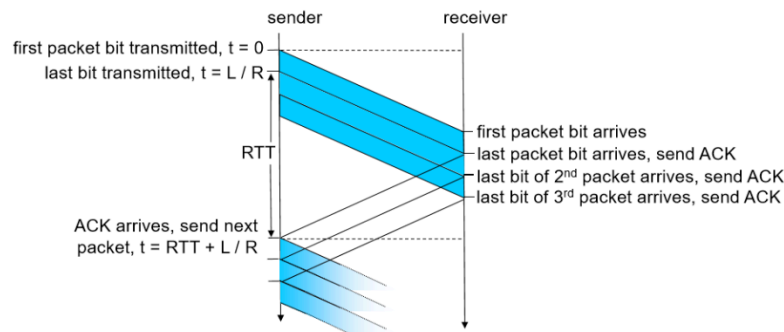
- Sender sends P3, and P4, P5, P6
- Receiver sends ACK 4



Note: **ACK- n** means *ready to receive frame # n* (i.e, the previous $n-1$ frames have been received without error).

Discussion of the Sliding Window

- **Achieves the goal of *Flow Control***
 - *The receiver can control the size of the sending window (by sending ACK).*
 - *Sending window: the frames that are allowed to be sent out (without waiting).*
- **Why Sliding Window: to improve efficiency**
 - *Pipelining: we want the sender to fill the “pipe” without stopping -> efficiency!*
 - **Bandwidth-delay Product**: the capacity of the “pipe” (channel)



$$\eta = \frac{k \cdot L/R}{S + k \cdot L/R} = \frac{k \cdot L}{S \cdot R + k \cdot L}$$

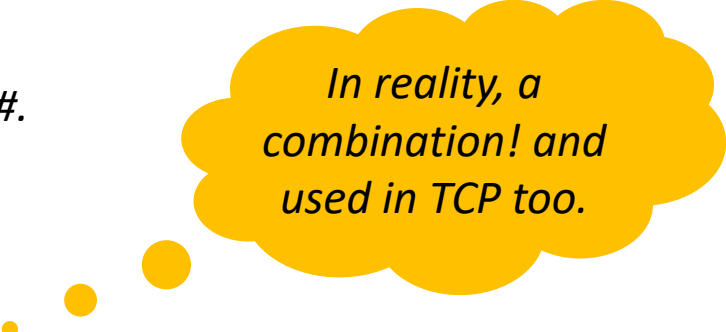
How to set the window size k ?

Go-Back-N (GBN) Overview (1/2)

- *If no error and no loss -> GBN = sliding window protocol*
- ***N** : **window size W**, How many successive frames can be sent without ACK*
- ***A timer** is set for each in-flight (transmitted but not ACKed) frame*
 - *Timeout value of the timer $T \geq W \cdot L/R$*
 - *L is the frame size, R is the data rate, and L/R is the transmission delay.*
- ***Operations:***
 - ***Sender:** transmits frames #0, 1, ..., **W-1** and waits for up to T s for each of their ACKs. As soon as it receives an ACK for frame #0, it sends frame #W.*

Go-Back-N (GBN) Overview (2/2)

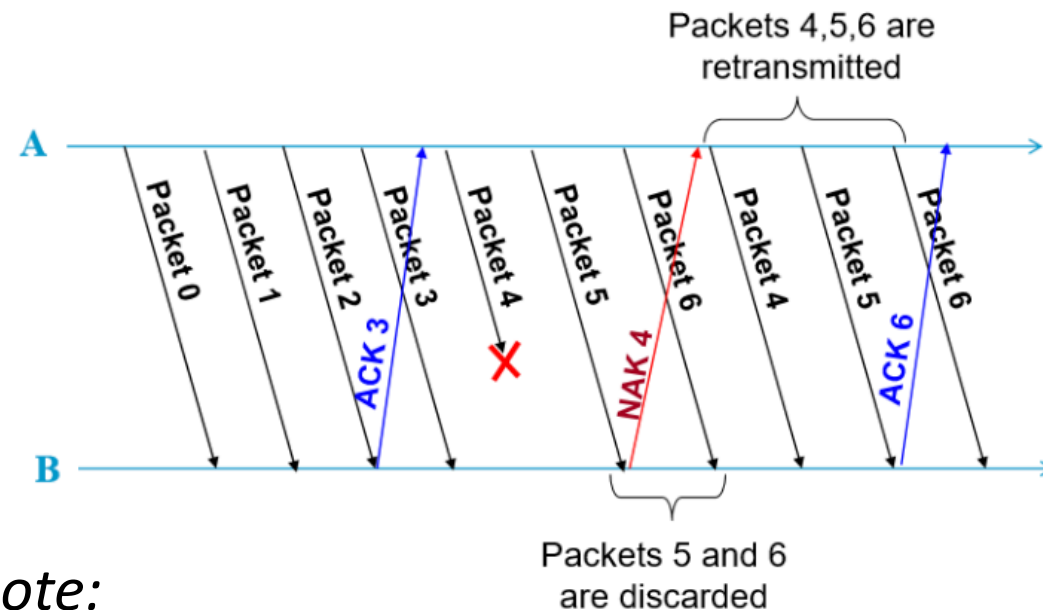
- **When something is wrong,**
 - Sender **timer timeout**, “go back” and retransmit all subsequent frames.
 - Receiver receives **out-of-order (or erroneous) frames** -> discard (no buffer)!
 - ACK-based solution:
 - Receiver: Re-ACK the frame with the highest in-order seq. #.
 - Sender may receive duplicate ACKs.
 - NAK-based solution:
 - Receiver sends NAK i if frame $\#i$ is erroneous. After this, it will discard all the frames until frame $\#i$ is retransmitted and correctly received.
 - Sender receives NAK i , and it will retransmit frame $\#i$ and all frames $\#i+1$, $\#i+2$, ... that have been sent, but not ACKed.



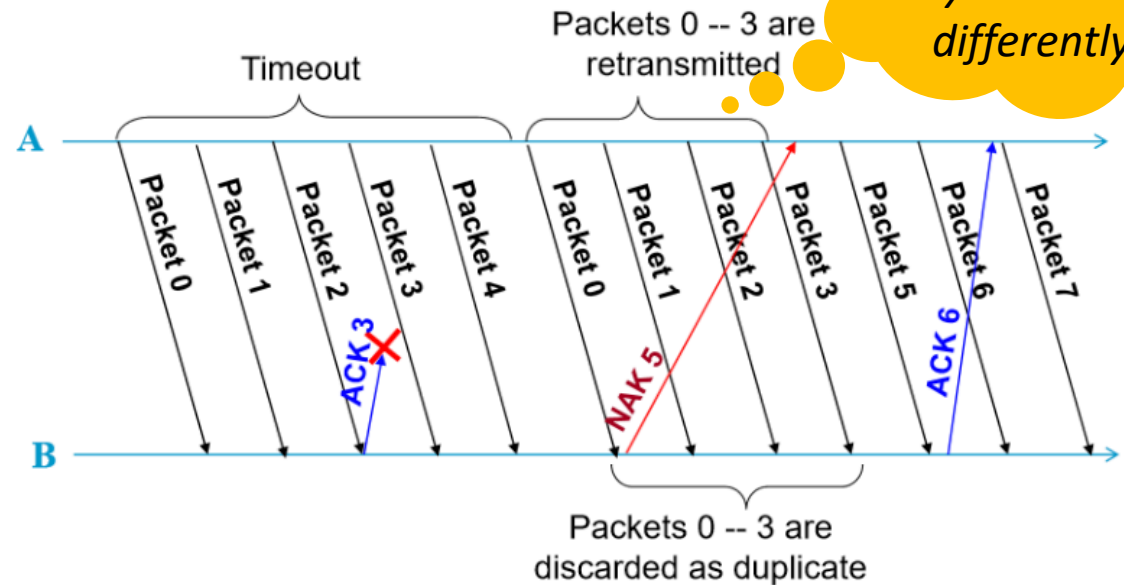
In reality, a combination! and used in TCP too.

Effects of Transmission Errors

• Case 1. Lost Frames/Packets



• Case 2. Lost ACK



In reality, out-of-order frames may be handled differently.

Note:

- **ACK-*n*** means *ready to receive frame #*n** (the previous *n-1* frames have been received without error).
- **NAK-*n*** means the previous received *frame #*n** is erroneous, but the previous *n-1* frames have been received without error.
- Erroneous: 1) in-order frames/packets but fails CRC; or 2) out-of-order frames/packets.

Go-Back-N Operations: A-to-B transmission

Sender A keeps a transmission timer.

1) Case 1. Damaged Frame

- a. **Erroneous:** A transmits frame i . B detects an error, with the previous $i-1$ frames successfully received.
 - Receiver B must send **NAK i** immediately.
 - When **Sender A** receives **NAK i** , it must **retransmit** frame i and all the subsequent frames it has transmitted.
- b. Frame i is **lost**, but B has received the subsequent frame $i+1$. B treats it as damaged frame i , and must send **NAK i** .
- c. Frame i is **lost**, and A did not send additional frames. B may return either an ACK i , or a NAK i . **Timer** at A for frame i will eventually **expire**.

2) Case 2. Lost ACK

- a. B receives frame i and sends **ACK $i+1$** , which is **lost**. A receives an ACK for subsequent frames, so timer for frame i will not expire, and everything is fine.
- b. **Timer** for frame i **expires**, sender A will **retransmit** frame i and subsequent frames.

3) Case 3. Lost NAK

If a **NAK i** is **lost**, A will time out on associated (later) frames, and **retransmit** frame i and all subsequent frames.

Note: Not every frame is ACKed individually.

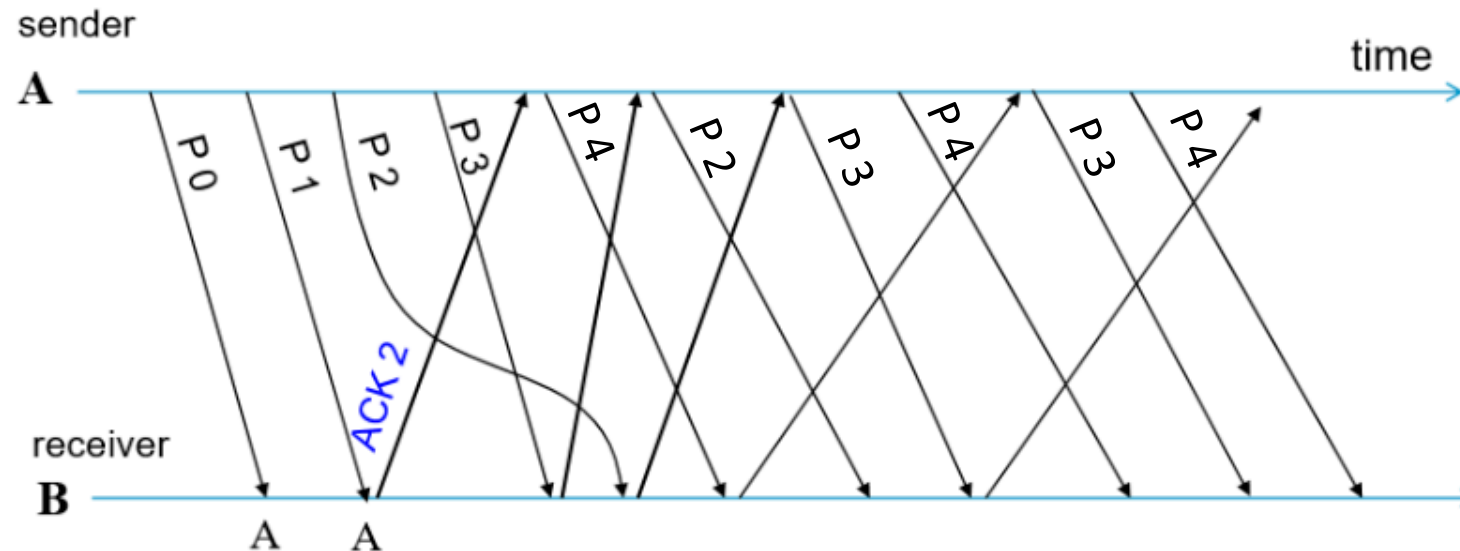
Ex.3.6 Go-Back-N

Consider A-to-B transmission with Go-Back-N protocol:

- Assume the window size is sufficiently large.

Fill in the blanks with

- Types and Numbers: $P\#$ (Packet/Frame #), ACK # or NAK #
- Possible actions of the receiver: **A**(Accept), **DE**(Discard as Error), **DD**(Discard as Duplicate)



Discussion on Window Size

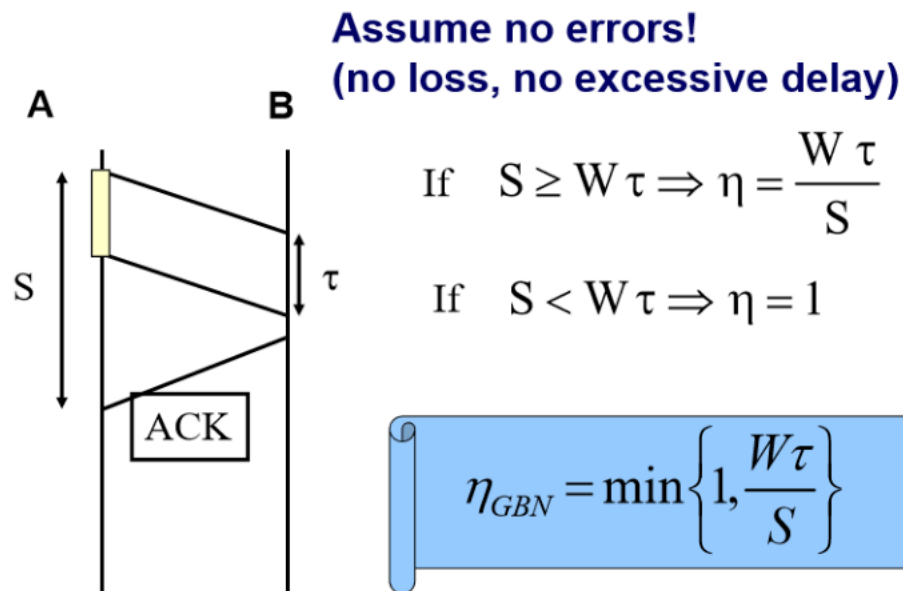
- **Window size W : a k -bit seq. # can support $W=2^k - 1$ GBN. (Not 2^k !)**
 - WHY: Consider **$W=8$** and $k=3$. A transmits $P0$ and gets $ACK1$, and then transmits $P1,2,3,4,5,6,7,0$. Now A gets another $ACK1$.
 - 1) Possibility 1: All the 8 frames/packets were successfully received by B.
 - 2) Possibility 2: All the 8 frames/packets were lost/erroneous, and B was repeating $ACK1$.
 - To avoid ambiguity, $W \leq 2^k - 1$. The max. number of outstanding frames/packets (i.e. window size) can not exceed $2^k - 1$.
- **Duplicated ACKs are use in some protocols to be “NAK-free”.**

Which one?
Sender A is
confused!

Efficiency of GBN

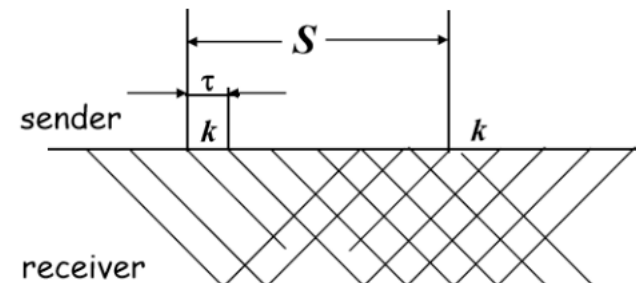
- Transmission delay $\tau = L/R$
- Window size W . Sender A transmits $P_0, P_1, \dots, P(W-1)$ and waits for the first ACK to arrive.
- Assume the ACK of each packet arrives exactly S seconds after the start of its transmission.

1) Case 1. No error, no delay.



2) Case 2. With errors, no delay.

- Errors reduce efficiency.
- Let $W = S/\tau$, the min. value s.t. $\eta = 1$.
- Transmission timeout $T = S = W\tau$



$$\eta_{GBN} = \frac{\tau}{\tau + (1-p)S/p}$$

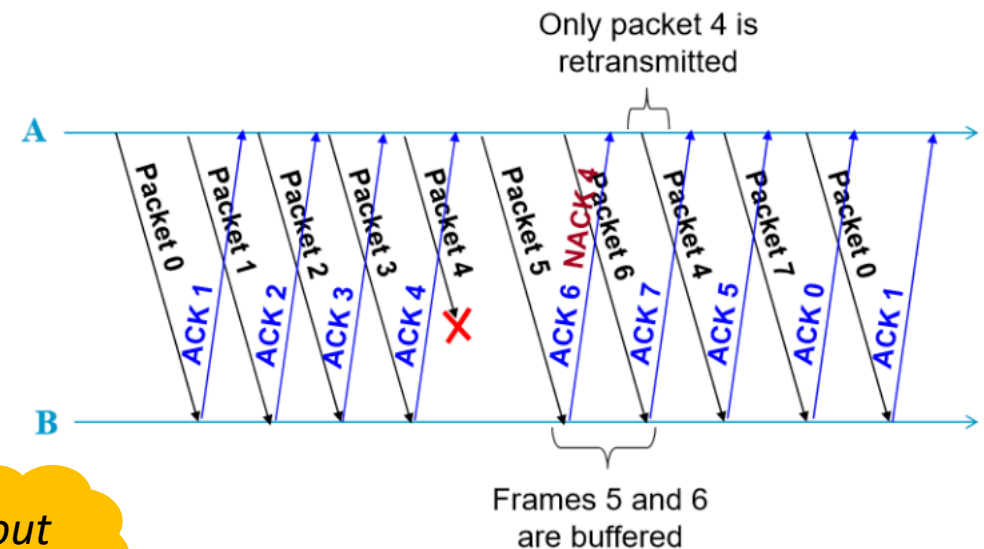
Selective Repeat

- Similar to GBN, but the sender only retransmits the frames for which the ACK is not received (or a NAK is received).
 - Advantage (over GBN): fewer retransmissions.
 - Disadvantages:
 - 1) More complicated protocol.
 - 2) Each frame must be **individually ACKed**.
 - 3) Needs **buffer** for out-of-order frames.
- Efficiency: same as GBN, if w/o errors

$$\eta_{SRP} = \min\left\{1, \frac{W\tau}{S}\right\}$$

How about w/ errors?

■ Lost Packet



Summary

Functions of the DLC sublayer: providing an **error-free** bit pipe

- **Framing:** bit stream -> sequence of frames
 - Count, flag bytes, byte/bit stuffing
- **Error Control Coding:** detect/correct bit errors
 - Capability of a Coding Scheme: **Hamming distance**
 - **Error-detecting Code**
 - **(Even) Parity Check**
 - **CRC** - $M(x)$, $G(x)$, the remainder
 - **Error-correcting Code**
 - **Hamming Code** - the table (data bits, check bits), the rules, bottom-up direction
- **ARQ:** Error recovery by retransmission
 - **Key:** How to tell the sender that something is wrong about a packet/frame?
 - **Sender:** handles lost frames -> keep a **timer** with timeout value T
 - **Receiver:** handles erroneous frames -> send out **ACK/NAK**
 - Stop-And-Wait (SAW): Seq. # (SN), ACK
 - Sliding Window: **Pipelining** -> increases efficiency by filling the 'pipe' (bandwidth-delay product).
 - **Original SW:** Sender Window, Receiver Window; Shrinks (left edge ->), Expands (right edge ->)
 - **Go-Back-N:** SW + error handling by P#, ACK#, NAK#, actions
 - **Selective Repeat:** Only retransmit the damaged frames by buffering out-of-order frames, and ACK for every frame

Will see it
again in TCP